

A New Upper Bound on the Binary Deletion Channel Capacity

Özgür Soysal

September 11, 2026

AI: disclosure

This work is an AI-assisted mathematics project. Claude Fable 5.1, GPT 5.6 Sol, and GPT 6 Astra were used in its development. The author is responsible for the manuscript and its claims.

Abstract

We prove that the capacity of the binary deletion channel satisfies $C(d) \leq (1 - d)/4$ for every $13/20 \leq d < 1$. The proof describes the output from right to left, using a six-bit context to assign a description length. We bound the increase in expected description length minus output entropy when one input bit is added. A relative-entropy identity reduces this bound to finitely many linear inequalities. A potential on input windows of length 26 makes the inequalities telescope, giving a bound for every input word. Deletion composition extends the result from $d = 13/20$ to all larger deletion probabilities. We also obtain finite-block bounds on mutual information and decoding error, with explicit $O(\log n/n)$ corrections. The finite certificate is checked using exact integer arithmetic, and the proof is formalized end to end in Lean.

Keywords. Binary deletion channel; capacity converse; predictive distributions; finite certificates; formal verification; Lean.

Contents

1	Introduction and statement of results	3
1.1	Relation to previous bounds	4
1.2	Outline of the proof	4
2	Channel model, finite laws, and operational capacity	4
2.1	Words and the channel kernel	4
2.2	Codes and achievable rates	5
3	Finite entropy inequalities used in the proof	5
4	Backward description lengths and a positive test law	6
5	Predictive laws and the entropy increment	7
6	A global affine majorant	8
7	Finite windows, reachable laws, and the Bellman certificate	8
8	Deletion composition and a general converse	9

9	The exact numerical certificate	11
9.1	Exact reference numerators and adjoints	11
9.2	Eliminating every real logarithm from the check	12
10	The certificate and completion of the proof	13
11	Consequences, limitations, and further work	14
A	Exact code lengths	14
B	Rational proof of the Kraft inequalities	16
C	The logarithm enclosure routine	17
C.1	Positive centers and directed rounding	17
C.2	Normalized grid	17
C.3	Thirty-two affine pieces	18
D	A scalar specification of the exhaustive check	19
E	Exact arithmetic and storage	19
E.1	Packed arithmetic	19
E.2	A single potential across all batches	20
E.3	Finite coverage and the concrete proposition	20
E.4	Identification of the electronic certificate	21
F	Formal correspondence and verification record	21
F.1	The five closed declarations	21
F.2	Verification record	22
F.3	Foundations and limits of the claim	22
G	Correspondence between mathematics and Lean	22
H	Reproduction and document provenance	23

1 Introduction and statement of results

The binary deletion channel independently removes each input bit with probability d and delivers the surviving subsequence in its original order. The receiver observes neither the deleted bits nor their positions. The resulting uncertainty about alignment makes even this elementary channel difficult to analyze. Cheraghchi and Ribeiro [1] survey the capacity theory of synchronization channels and the methods developed for them.

The present paper gives a converse in the high-deletion regime by following the probability law of a short prefix of the future output. This law retains enough alignment information to control the increase of output entropy when one input bit is prepended. A backward description cost uses the same law. Their difference is a local reward. We dominate this reward by a finite family of affine functionals and certify a potential inequality over every input-window state. The finite certificate controls all remote suffix laws, not just the reference distributions used to construct it.

The result is stated operationally, for uniform-message average-error codes with arbitrary deterministic encoders and decoders. Section 2 fixes these definitions. Entropy and mutual information in displayed information expressions are measured in nats; rates and capacity are in bits per input bit.

Theorem 1.1 (Quarter bound and finite-block converses). *For every $d \in [13/20, 1)$ and every positive integer n :*

(i) *Every law of $X \in \{0, 1\}^n$, with channel output Y , satisfies*

$$\frac{I(X; Y)}{n \log 2} \leq \frac{1-d}{4} + \frac{\log_2(n+1) + 1}{n}. \quad (1.1)$$

(ii) *Every length- n block code of rate R_n and average error P_e satisfies*

$$(1 - P_e)R_n \leq \frac{1-d}{4} + \frac{\log_2(n+1) + 2}{n}. \quad (1.2)$$

(iii) *The operational capacity satisfies*

$$C(d) \leq \frac{1-d}{4}. \quad (1.3)$$

(iv) *If $n_j \rightarrow \infty$ and length- n_j codes have $P_{e,j} \rightarrow 0$, then for every $\varepsilon > 0$, all sufficiently large j satisfy*

$$R_{n_j} < \frac{1-d}{4} + \varepsilon. \quad (1.4)$$

In particular, any limit of those rates is at most $(1-d)/4$.

The proof has two parts. First, Theorem 8.2 shows how a finite collection of inequalities bounds the capacity. Then Proposition 10.1 supplies an explicit certificate satisfying those inequalities. The argument leading from the certificate to the capacity bound is given in full below.

Corollary 1.2. *The high-deletion normalized capacity obeys*

$$\limsup_{d \uparrow 1} \frac{C(d)}{1-d} \leq \frac{1}{4}.$$

Proof. For $13/20 \leq d < 1$, divide (1.3) by the positive number $1-d$ and take the upper limit. $\square$

This corollary does not require an existence theorem for the normalized limit. Dalai [2] proved that the limit exists and equals the infimum of the normalized capacity; invoking that external theorem gives the same upper bound on the limit itself.

1.1 Relation to previous bounds

Rubinstein and Con [3] obtained the upper bound $0.3745(1 - d)$ for $d \geq 0.68$ by reducing the space requirements of Blahut–Arimoto computations [4, 5]. Pinto and Ribeiro [6] subsequently obtained $0.3578(1 - d)$ for $d \geq 0.64$ using a parallelized implementation. The quarter bound improves the Rubinstein–Con coefficient for $d \geq 0.68$ and the Pinto–Ribeiro coefficient for $d \geq 0.65$. The present construction uses an input-window state space and a fixed predictive output context, rather than the entire transition matrix of a finite-block subsequence channel.

Auxiliary output distributions are standard tools in divergence-based capacity converses; see, for example, the synchronization-channel applications surveyed in [1, Sec. 4]. Here the specific backward output law and the global finite certificate give a bound uniform over all input words and all remote suffix laws.

1.2 Outline of the proof

The main quantity is expected description length minus output entropy. Section 4 explains why an upper bound on this quantity gives a capacity converse. Sections 5 and 6 bound its increase when a bit is prepended. Section 7 reduces the problem to a finite inequality and shows how a potential makes the local bounds telescope. Section 8 then derives the information, decoding, and capacity bounds.

The remaining step is an explicit computer-assisted calculation. Sections 9 and 10 specify the integer inequalities and the certificate that satisfies them. Its potential has 2^{26} entries and is supplied electronically. The complete proof, including this finite calculation, is formalized in Lean; the appendices give the implementation details and verification record.

2 Channel model, finite laws, and operational capacity

2.1 Words and the channel kernel

Let $\{0, 1\}^0 = \{\emptyset\}$ and $\mathcal{Y}_n = \bigsqcup_{k=0}^n \{0, 1\}^k$; the disjoint union records the word length. For $x = x_1 \cdots x_n$ and $e \in \{0, 1\}^n$, let x_e be the subsequence of coordinates with $e_i = 1$, and let $|e| = \sum_i e_i$. Define

$$W_d(y \mid x) = \sum_{e \in \{0, 1\}^n : x_e = y} (1 - d)^{|e|} d^{n - |e|}, \quad y \in \mathcal{Y}_n. \quad (2.1)$$

As usual, exponent zero contributes the factor one, including at an endpoint of $[0, 1]$. All summands are nonnegative and $\sum_y W_d(y \mid x) = (d + (1 - d))^n = 1$. Distinct masks yielding the same word are summed, not identified with a single mask.

For a finite set A , write $\mathcal{P}(A)$ for its probability simplex. For $P \in \mathcal{P}(A)$ and a map $f : A \rightarrow B$, the pushforward is $(f_{\#}P)(b) = \sum_{a: f(a)=b} P(a)$. We use

$$H(P) = - \sum_{a: P(a) > 0} P(a) \log P(a), \quad D(P \parallel Q) = \sum_{a: P(a) > 0} P(a) \log \frac{P(a)}{Q(a)}$$

when $\text{supp } P \subseteq \text{supp } Q$. No expression $\log 0$ is needed in this convention. All uses of divergence below either have a strictly positive second law or explicitly verify the support inclusion.

If X has law P and channel W , its output law is $P_Y = \sum_x P(x) W_x$, and

$$I(X; Y) = \sum_x P(x) D(W_x \parallel P_Y) = H(P_Y) - \sum_x P(x) H(W_x).$$

Terms with $P(x) = 0$ can be omitted, so the divergence support condition holds wherever it is used.

2.2 Codes and achievable rates

A block code of input length n is a positive integer M , an encoder $f : \{1, \dots, M\} \rightarrow \{0, 1\}^n$, and a decoder $g : \mathcal{Y}_n \rightarrow \{1, \dots, M\}$. The encoder need not be injective. For a uniform message U , set $X = f(U)$ and

$$P_e = \mathbb{P}\{g(Y) \neq U\}, \quad R_n = \frac{\log M}{n \log 2} \quad (n > 0).$$

A nonnegative rate R is achievable if there is such a code at every input length, its error tends to zero, and its rate tends to R . We define $C(d)$ as the supremum of these rates. The one-message code shows that the achievable set contains zero. The upper bound proved below shows that this set is bounded above.

Our finite-block statements concern positive lengths. The conclusion (1.4) also applies to codes defined along any sequence of lengths tending to infinity, so the converse does not depend on requiring a code at every length.

3 Finite entropy inequalities used in the proof

The chain rule, log-sum inequality, convexity of relative entropy, and data-processing inequality are classical; see Cover and Thomas [7, Ch. 2]. We include their finite-law proofs to make the support conditions explicit.

For finite random variables A, B , define

$$H(A | B) = \sum_{b: \mathbb{P}(B=b) > 0} \mathbb{P}(B = b) H(\mathcal{L}(A | B = b)).$$

Conditional laws on zero-probability events do not enter this sum. On every positive joint-mass coordinate, $\mathbb{P}(A = a, B = b) = \mathbb{P}(B = b)\mathbb{P}(A = a | B = b)$. Taking logarithms, multiplying by joint mass, and summing gives the chain rule $H(A, B) = H(B) + H(A | B)$. Subtracting the two chain-rule forms gives $I(A; B) = H(A) - H(A | B) = H(B) - H(B | A)$. The same calculation conditional on a third variable gives the conditional chain rule used in the decoding argument.

Lemma 3.1 (Log-sum inequality with support conditions). *Suppose $a_i, b_i \geq 0$ on a finite index set, and $b_i = 0$ implies $a_i = 0$. Put $A = \sum_i a_i$ and $B = \sum_i b_i$. If $A > 0$, then $B > 0$ and*

$$A \log(A/B) \leq \sum_{i: a_i > 0} a_i \log(a_i/b_i). \quad (3.1)$$

If $A = 0$, both sides are interpreted as zero and the inequality remains true.

Proof. If $A = 0$, every a_i vanishes. Otherwise $B > 0$ by the support condition. Set $p_i = a_i/A$ and $q_i = b_i/B$. For $p_i > 0$, the elementary inequality $\log t \leq t - 1$, applied to $t = q_i/p_i$, gives $p_i \log(p_i/q_i) \geq p_i - q_i$. Summing over $p_i > 0$ gives a lower bound $1 - \sum_{p_i > 0} q_i \geq 0$. Multiplication by A and expansion of the logarithm give (3.1). $\square$

Lemma 3.2 (Divergence contraction and mixing). *Under the required support inclusions:*

- (i) $D(P \| Q) \geq 0$;
- (ii) $D(f_{\#}P \| f_{\#}Q) \leq D(P \| Q)$ for every map f ;

(iii) for probability weights λ_j ,

$$D\left(\sum_j \lambda_j P_j \parallel \sum_j \lambda_j Q_j\right) \leq \sum_j \lambda_j D(P_j \parallel Q_j).$$

Proof. Part (i) is Lemma 3.1 with totals one. For (ii), apply that lemma on each fiber $f^{-1}(b)$ and sum over b . For (iii), at each coordinate apply it to $a_j = \lambda_j P_j(a)$ and $b_j = \lambda_j Q_j(a)$, then sum over a . A zero mixture weight contributes zero and causes no exceptional case. $\square$

In particular, a law on an alphabet of size $k \geq 1$ has entropy at most $\log k$: its divergence from the uniform law is $\log k - H(P) \geq 0$. Applying this to each conditional law also bounds conditional entropy by $\log k$. If a random variable is determined by the variables being conditioned on, that conditional entropy is zero because every contributing law is a point mass.

Lemma 3.3 (Information and deterministic processing). *For finite random variables B, Z and a function v , $I(B; v(Z)) \leq I(B; Z)$. Also, if Q is a positive law on the output alphabet of a finite channel, then*

$$\sum_x P(x) D(W_x \parallel Q) = I(X; Y) + D(P_Y \parallel Q). \quad (3.2)$$

Proof. For the first assertion, apply Lemma 3.2(ii) to the joint law of (B, Z) and the product of its marginals, under $(b, z) \mapsto (b, v(z))$. The support condition holds since positive joint mass implies positive marginal masses. For the second assertion, expand the left side and add and subtract $\sum_y P_Y(y) \log P_Y(y)$. Coordinates with $P_Y(y) = 0$ have zero mass in every positively weighted W_x , so the expansion is valid on the indicated supports. $\square$

4 Backward description lengths and a positive test law

We first construct a reference distribution on output words. Its negative logarithm is a description length, up to a normalizing constant. Bounding expected description length minus entropy will therefore bound divergence from this reference distribution.

Let $\mathcal{U} = \{0, 1, \dots, 63\}$. For $c \in \{0, 1\}$, define

$$\sigma_c(i) = (2i + c) \bmod 64.$$

Define the context recursively by $u(\emptyset) = 0$ and $u(cy) = \sigma_c(u(y))$. Equivalently, for $y = y_1 \cdots y_k$,

$$u(y) = \sum_{j=1}^{\min(k, 6)} 2^{j-1} y_j. \quad (4.1)$$

Missing coordinates are zero. This statistic may identify a short word with a longer word ending in zeros; injectivity of u is not assumed.

The Kraft inequality is the standard normalization condition for prefix-code lengths [7, Ch. 5]. We use its real-length form: fix $w_{i,c}$ satisfying the row-wise inequalities

$$e^{-w_{i,0} \log 2} + e^{-w_{i,1} \log 2} \leq 1 \quad (i \in \mathcal{U}). \quad (4.2)$$

Define $L(\emptyset) = 0$ and

$$L(cy) = w_{u(y),c} + L(y). \quad (4.3)$$

These real description lengths need not be lengths of literal binary codewords. Only the positive weights $2^{-L(y)}$ and their partition bound enter the proof.

Lemma 4.1 (Partition bound). *For $Z_n = \sum_{y \in \mathcal{Y}_n} 2^{-L(y)}$, one has $1 \leq Z_n \leq n + 1$. Thus $Q_n(y) = 2^{-L(y)} / Z_n$ is a strictly positive probability law on $\mathcal{Y}_n$ and, for every $P \in \mathcal{P}(\mathcal{Y}_n)$,*

$$D(P \| Q_n) \leq (\log 2) \mathbb{E}_P L - H(P) + \log(n + 1). \quad (4.4)$$

Proof. The empty word contributes one. Every nonempty word of length at most $n + 1$ is uniquely cy with $y \in \mathcal{Y}_n$. Therefore

$$Z_{n+1} = 1 + \sum_{y \in \mathcal{Y}_n} 2^{-L(y)} (2^{-w_{u(y),0}} + 2^{-w_{u(y),1}}) \leq 1 + Z_n.$$

Induction from $Z_0 = 1$ gives the upper bound. The divergence identity

$$D(P \| Q_n) = (\log 2) \mathbb{E}_P L - H(P) + \log Z_n$$

then proves (4.4). In particular, the test law includes the empty output and every possible output length. $\square$

5 Predictive laws and the entropy increment

Fix temporarily $0 < d < 1$. For a law μ on $\mathcal{U}$, put

$$F_c \mu = d\mu + (1 - d)(\sigma_c)_{\#} \mu. \quad (5.1)$$

For a word $x = x_1 \cdots x_k$, set $F_x = F_{x_1} \circ \cdots \circ F_{x_k}$, with the rightmost operator acting first; $F_{\emptyset}$ is the identity. Let p_x be the law of $u(Y)$ under $W_d(\cdot | x)$.

Lemma 5.1 (Exact recursion). *$p_{\emptyset} = \delta_0$, $p_{cx} = F_c p_x$, and $p_x = F_x \delta_0$.*

Proof. Let Y be the deletion output from x , and let B independently equal one with probability $1 - d$. The deletion output from cx is Y if $B = 0$ and cY if $B = 1$. Equation (4.1) gives the recursion. Induction proves the final identity. $\square$

Define the weighted Jensen–Shannon divergence [8]

$$J_d(\mu, \nu) = H(d\mu + (1 - d)\nu) - dH(\mu) - (1 - d)H(\nu).$$

It is the mutual information between a binary mixture label and the resulting observation. Define the score and local reward, in nats, by

$$S_d(x) = (\log 2) \mathbb{E}[L(Y) | x] - H(W_d(\cdot | x)), \quad (5.2)$$

$$r_c(\mu) = (1 - d)(\log 2) \sum_i \mu_i w_{i,c} - J_d(\mu, (\sigma_c)_{\#} \mu). \quad (5.3)$$

Lemma 5.2 (One-symbol score bound). *$S_d(\emptyset) = 0$ and $S_d(cx) - S_d(x) \leq r_c(p_x)$.*

Proof. Use the variables Y, B in Lemma 5.1, and call the new output Z . Conditional on $B = 0$, the map $Y \mapsto Z$ is the identity; conditional on $B = 1$, it prepends c . Both are injective on the old output alphabet, so $H(Z | B) = H(Y)$. Consequently

$$H(Z) - H(Y) = I(B; Z) \geq I(B; u(Z)) = J_d(p_x, (\sigma_c)_{\#} p_x).$$

Independence and (4.3) give $\mathbb{E}L(Z) - \mathbb{E}L(Y) = (1 - d) \sum_i (p_x)_i w_{i,c}$. Subtract the entropy inequality from this cost identity. For the empty input both the output entropy and its description length are zero. $\square$

Remark 5.3. The laws of Y and cY can overlap when embedded in the larger output alphabet. The proof does not add a binary entropy term as though their supports were disjoint. It uses the actual mutual information of the retention label. This distinction is necessary for the inequality to hold.

6 A global affine majorant

The reward depends nonlinearly on the predictive law. We now bound it by a linear function of that law. This will allow us to check the bound on a finite collection of distributions.

Let $q \in \mathcal{P}(\mathcal{U})$ be strictly positive. Define a vector $g_c(q) \in \mathbb{R}^{64}$ by

$$\begin{aligned} g_c(q)_i &= (1-d)(\log 2)w_{i,c} + d \log(F_c q)_i + (1-d) \log(F_c q)_{\sigma_c(i)} \\ &\quad - d \log q_i - (1-d) \log((\sigma_c)_\# q)_{\sigma_c(i)}. \end{aligned} \quad (6.1)$$

Every logarithm displayed here is of a positive number: $F_c q \geq dq > 0$, and $((\sigma_c)_\# q)_{\sigma_c(i)} \geq q_i > 0$.

Lemma 6.1 (Exact majorant slack). *For every $\mu \in \mathcal{P}(\mathcal{U})$,*

$$\begin{aligned} \langle \mu, g_c(q) \rangle - r_c(\mu) &= d D(\mu \| q) + (1-d) D((\sigma_c)_\# \mu \| (\sigma_c)_\# q) \\ &\quad - D(F_c \mu \| F_c q) \geq 0. \end{aligned} \quad (6.2)$$

In particular, $r_c(\mu) \leq \langle \mu, g_c(q) \rangle$, with equality at $\mu = q$.

Proof. The support condition for the second divergence holds because a zero coordinate of $(\sigma_c)_\# q$, with q positive, has an empty preimage under σ_c ; the corresponding pushforward of μ is also zero. The other denominator laws are positive.

Write $\nu = (\sigma_c)_\# \mu$ and $v = (\sigma_c)_\# q$. By the pushforward summation identity, expansion of (6.1) yields

$$\begin{aligned} \langle \mu, g_c(q) \rangle &= (1-d)(\log 2) \langle \mu, w_{\cdot,c} \rangle + \sum_i (F_c \mu)_i \log(F_c q)_i \\ &\quad - d \sum_i \mu_i \log q_i - (1-d) \sum_{i: \nu_i > 0} \nu_i \log v_i. \end{aligned}$$

Subtract (5.3) and expand the three entropies to obtain the equality in (6.2). Lemma 3.2(iii), applied to the two mixture components, makes the right side nonnegative. At $\mu = q$ all three divergences vanish. $\square$

This proof is global on the closed simplex. It does not extend a derivative formula to its boundary without justification. In particular, the actual predictive law of a short or constant input can have zeros; the reference law alone needs full support.

7 Finite windows, reachable laws, and the Bellman certificate

For a vector $v \in \mathbb{R}^{64}$ let $(B_c v)_i = dv_i + (1-d)v_{\sigma_c(i)}$. Define A_x by

$$A_\emptyset = I, \quad A_{cx} = A_x B_c.$$

Thus, if $x = x_1 \cdots x_k$, then $A_x = B_{x_k} \cdots B_{x_1}$. In evaluating $A_x v$, the operation B_{x_1} is applied first. This is the reverse composition order to the forward action on probability laws.

Lemma 7.1 (Adjoint and positivity). $\langle F_x \mu, v \rangle = \langle \mu, A_x v \rangle$. Moreover $A_x \mathbf{1} = \mathbf{1}$, and $v \leq v'$ coordinatewise implies $A_x v \leq A_x v'$ coordinatewise.

Proof. The one-step identity follows by reindexing the pushforward term: $\sum_j ((\sigma_c)_\# \mu)_j v_j = \sum_i \mu_i v_{\sigma_c(i)}$. Iterate it in the stated order. Each B_c is a nonnegative weighted average with weights summing to one, so it preserves constants and order; its compositions do as well. $\square$

Fix a positive window length m . Let $s(x) \in \{0, 1\}^m$ be the first m bits of x , padded on the right with zeros if $|x| < m$. Let $s^+(s, c)$ be the first m bits of cs . Define

$$\mathbf{u} = (1/64, \dots, 1/64), \quad q_s = F_s \mathbf{u}.$$

Every coordinate of q_s is at least $d^m/64 > 0$.

Lemma 7.2 (Every actual law is represented). *For each finite word x , there is a law μ such that $p_x = F_{s(x)} \mu$. Also $s(cx) = s^+(s(x), c)$.*

Proof. If $|x| \geq m$, write $x = s(x)z$ and take $\mu = p_z$. If $|x| = k < m$, take $\mu = \delta_0$ and note that $F_0 \delta_0 = d \delta_0 + (1-d) \delta_{\sigma_0(0)} = \delta_0$. Thus $F_{x0^{m-k}} \delta_0 = F_x \delta_0 = p_x$. The update identity follows by checking truncation and padding directly in the two length cases. $\square$

Definition 7.3 (Finite certificate). A potential $h : \{0, 1\}^m \rightarrow \mathbb{R}$ and number θ form a certificate for (w, d, m) if, for every $s \in \{0, 1\}^m$, $c \in \{0, 1\}$, and $i \in \mathcal{U}$,

$$(A_s g_c(q_s))_i + h(s^+(s, c)) - h(s) \leq \theta. \quad (7.1)$$

Lemma 7.4 (From coordinates to arbitrary suffixes). *If (7.1) holds, then for every finite word x and bit c ,*

$$r_c(p_x) + h(s(cx)) - h(s(x)) \leq \theta.$$

Proof. By Lemma 7.2, $p_x = F_s \mu$ for $s = s(x)$ and a law μ . Lemmas 6.1 and 7.1 imply $r_c(p_x) \leq \langle \mu, A_s g_c(q_s) \rangle$. Multiply (7.1) by $\mu_i \geq 0$ and sum over i ; its potential and constant terms are unchanged because $\sum_i \mu_i = 1$. Use the window update identity. $\square$

Proposition 7.5 (Uniform score bound). *If (7.1) holds and $0 \leq h(s) \leq B$ for every state, then*

$$S_d(x) + h(s(x)) \leq |x|\theta + h(0^m), \quad S_d(x) \leq |x|\theta + B. \quad (7.2)$$

Proof. The first assertion follows by induction on word length. It is equality for the empty word. In the induction step add Lemma 5.2 and Lemma 7.4, and apply the induction hypothesis to the suffix. The second assertion follows from $h(0^m) - h(s(x)) \leq B$. Equivalently, one can prepend the bits from right to left and cancel the consecutive potential terms. The same proof works for words shorter than m ; no initial window is discarded. $\square$

8 Deletion composition and a general converse

Rahmati and Duman [9] proved the capacity inequality $C(\lambda d_0 + 1 - \lambda) \leq \lambda C(d_0)$ for $0 \leq \lambda \leq 1$. The normalized extension used here has the same form. We prove the underlying channel composition directly and retain the finite-block correction by using one common auxiliary output law.

Lemma 8.1 (Composition of deletion channels). *Fix $0 < d_0 < 1$ and $d_0 \leq d < 1$, and put $\alpha = (d - d_0)/(1 - d_0)$. A deletion channel with probability α , followed independently by a deletion channel with probability d_0 , equals W_d . The intermediate word Z has $\mathbb{E}[|Z| \mid x] = (1 - \alpha)|x|$.*

Proof. $0 \leq \alpha < 1$ and $(1 - \alpha)(1 - d_0) = 1 - d$. For each original coordinate independently draw a first retention bit with probability $1 - \alpha$ and a second with probability $1 - d_0$. Only the second bits attached to first-stage survivors are used. This constructs the same law as assigning independent second retention bits to the intermediate word. The products of the two bits are independent across original coordinates, with retention probability $1 - d$. Their subsequence therefore has kernel (2.1). The mean-length identity is the sum of the $|x|$ first-stage retention probabilities. $\square$

Theorem 8.2 (General finite-certificate converse). *Let $0 < d_0 < 1$, let w satisfy (4.2), and suppose (7.1) holds at d_0 with $0 \leq h \leq B$ and $\theta \geq 0$. Then for $d_0 \leq d < 1$, $n \geq 1$, and every input word $x \in \{0, 1\}^n$,*

$$D(W_d(\cdot \mid x) \parallel Q_n) \leq \frac{1-d}{1-d_0} n\theta + B + \log(n+1). \quad (8.1)$$

Consequently every input distribution and every block code satisfy

$$\frac{I(X; Y)}{n \log 2} \leq \frac{1-d}{1-d_0} \frac{\theta}{\log 2} + \frac{B/\log 2 + \log_2(n+1)}{n}, \quad (8.2)$$

$$(1 - P_e) R_n \leq \frac{1-d}{1-d_0} \frac{\theta}{\log 2} + \frac{B/\log 2 + \log_2(n+1) + 1}{n}. \quad (8.3)$$

Proof. First consider any intermediate word z of length $k \leq n$. Its second-stage output law naturally embeds in $\mathcal{Y}_n$. Embedding preserves its entropy and expected cost: it only adds zero-mass coordinates. Lemma 4.1 and Proposition 7.5, at d_0 , give

$$D(W_{d_0}(\cdot \mid z) \parallel Q_n) \leq k\theta + B + \log(n+1).$$

Crucially, the same positive law Q_n is used for every value of k . By Lemma 8.1, $W_d(\cdot \mid x)$ is the mixture of these laws over $Z \sim W_\alpha(\cdot \mid x)$. Apply convexity in the first argument of divergence, Lemma 3.2(iii), and then the mean-length identity. This proves (8.1).

Apply (3.2), discard the nonnegative output divergence, and use (8.1) to obtain (8.2). For the decoding assertion we use the elementary form of Fano's inequality [7, Ch. 2], whose short proof follows. Let U be the uniform message, $\hat{U} = g(Y)$, and $E = \mathbf{1}_{\{U \neq \hat{U}\}}$. Since E is determined by (U, Y) , the conditional entropy chain rule gives

$$\begin{aligned} H(U \mid Y) &= H(E \mid Y) + H(U \mid E, Y) \\ &\leq \log 2 + P_e \log M. \end{aligned}$$

Indeed, when $E = 0$ the message is determined by Y , and when $E = 1$ its conditional entropy is at most $\log M$. This argument also holds for $M = 1$. Therefore

$$(1 - P_e) \log M \leq I(U; Y) + \log 2.$$

Equation (3.2) applied to the message channel bounds $I(U; Y)$ by the right side of (8.1), since each message selects one input word. Divide by $n \log 2$. $\square$

Corollary 8.3 (Operational and sequence consequences). *Under the hypotheses of Theorem 8.2, set $a_d = (1 - d)\theta/((1 - d_0) \log 2)$. Then $C(d) \leq a_d$. Every sequence of reliable codes with lengths tending to infinity has, for every $\varepsilon > 0$, eventual rate strictly below $a_d + \varepsilon$.*

Proof. Let $b_n = (B/\log 2 + \log_2(n+1) + 1)/n$. The elementary limit $\log(n+1)/n \rightarrow 0$ gives $b_{n_j} \rightarrow 0$. Since $P_{e,j} \rightarrow 0$, eventually $P_{e,j} < 1$, and (8.3) gives

$$R_{n_j} \leq \frac{a_d + b_{n_j}}{1 - P_{e,j}} \rightarrow a_d \quad \text{on the right-hand side.}$$

This proves the eventual assertion without assuming that rates converge or are bounded in advance. For any achievable rate apply it to the all-length sequence, then take the rate limit. The achievable set is nonempty and bounded above by a_d , so its supremum is at most a_d . $\square$

Remark 8.4 (What kind of converse is proved). The finite-block inequality gives, for $R_n > 0$, $P_e \geq 1 - (a_d + b_n)/R_n$. In particular, if $R_{n_j} \rightarrow R > a_d$, then $\liminf_j P_{e,j} \geq 1 - a_d/R > 0$. This elementary consequence does not say that the error tends to one; a strong converse is not asserted.

9 The exact numerical certificate

It remains to supply a potential and code lengths satisfying the finite inequality. We describe the chosen values and reduce the required inequalities to integer comparisons. The reduction accounts explicitly for the errors in the logarithm bounds.

We now specialize to

$$d_0 = \frac{13}{20}, \quad m = 26, \quad \theta = \frac{7}{80} \log 2, \quad B = \log 2. \quad (9.1)$$

The code lengths are $w_{i,c} = N_{i,c}/D$ with $D = 2^{55}$. All 128 integer numerators are given in Appendix A. The potential is

$$h(s) = \frac{\log 2}{P} H(s), \quad P = 65536, \quad H(s) \in \{0, \dots, 65535\}. \quad (9.2)$$

Thus $0 \leq h(s) < \log 2$. Section 10 specifies the state indexing, and Appendix E.4 identifies the electronic data.

9.1 Exact reference numerators and adjoints

For an integer vector $v \in \mathbb{N}^{64}$ define

$$(M_c v)_j = \sum_{i: \sigma_c(i)=j} v_i, \quad (G_c v)_j = 13v_j + 7(M_c v)_j, \quad (9.3)$$

$$(T_c v)_i = 13v_i + 7v_{\sigma_c(i)}. \quad (9.4)$$

The fibers of σ_c are explicit:

$$(M_c v)_j = \begin{cases} v_{\lfloor j/2 \rfloor} + v_{\lfloor j/2 \rfloor + 32}, & j \bmod 2 = c, \\ 0, & j \bmod 2 \neq c. \end{cases}$$

Starting with $v = \mathbf{1}$, process the bits of $s = s_1 \cdots s_m$ from s_m to s_1 , replacing v by $G_{s_j} v$. The resulting vector q^{int} satisfies

$$q_s = \frac{q^{\text{int}}}{64 \cdot 20^m}, \quad q_i^{\text{int}} \geq 13^m > 0, \quad \sum_i q_i^{\text{int}} = 64 \cdot 20^m. \quad (9.5)$$

These identities follow inductively from (9.3), since pushforward preserves total mass. For an action c , write $t = G_c q^{\text{int}}$ and $v_j = q_j^{\text{int}} + q_{j+32}^{\text{int}}$ for $0 \leq j < 32$. The integer adjoint is $T_s = T_{s_m} \cdots T_{s_1} = 20^m A_s$.

9.2 Eliminating every real logarithm from the check

Let $S = 2^{40}$. The certified logarithm functions used by the checker return integers $\ell(a), u(a)$ satisfying

$$\frac{\ell(a)}{S} \leq \log a \leq \frac{u(a)}{S} \quad (0 < a < 2^{144}). \quad (9.6)$$

Their complete finite description and soundness argument are given in Appendix C. We also use the exact constants

$$U_2 = 762123384787, \quad L_{20} = 3293842468468, \quad (9.7)$$

for which $(U_2 - 3)/S \leq \log 2 \leq U_2/S$ and $L_{20}/S \leq \log 20$. Set $E = 2^{112}$ and $Q = 20DS$. For each state and action define

$$a_i = E + 7N_{i,c}U_2 + D \left(13u(t_i) + 7u(t_{\sigma_c(i)}) - 13\ell(q_i^{\text{int}}) - 7\ell(v_{i \bmod 32}) - 20L_{20} \right). \quad (9.8)$$

Although this expression is written with integer subtraction, its value is nonnegative. The exact arithmetic bounds give $0 \leq a_i < 2^{113}$; Appendix E explains their role in the computation.

Lemma 9.1 (Integer majorant). *For every state, action, and coordinate, $g_c(q_s)_i \leq (a_i - E)/Q$.*

Proof. Substitute (9.5) into (6.1). The four occurrences of the reference denominator cancel, and the extra factor 20 in $F_c q_s$ contributes $-\log 20$. Hence

$$20g_c(q_s)_i = 7w_{i,c} \log 2 + 13 \log t_i + 7 \log t_{\sigma_c(i)} - 13 \log q_i^{\text{int}} - 7 \log v_{i \bmod 32} - 20 \log 20.$$

Every logarithm argument is positive. Use upper enclosures for the positive coefficients and lower enclosures for the negative coefficients. The fixed $w_{i,c}$ are nonnegative, so their $\log 2$ factor uses U_2 . Multiplication by DS gives the claim. The argument bounds $q_i^{\text{int}}, t, v < 2^{144}$, for example by their total masses $64 \cdot 20^{m+1}$ and $m \leq 28$; thus all logarithm enclosures apply. $\square$

For current and successor potential values H, H' , put

$$\Delta = 80(H' - H) - 7P, \quad b(H, H') = E - 2^{37}(\Delta U_2 + 3 \cdot 2^{23}). \quad (9.9)$$

The exhaustive integer predicate is

$$(T_s a)_i \leq 20^m b(H(s), H(s^+(s, c))) \quad (s \in \{0, 1\}^m, c \in \{0, 1\}, i \in \mathcal{U}). \quad (9.10)$$

Lemma 9.2 (Acceptance implies the Bellman inequality). *If (9.10) holds, then (7.1) holds with the parameters (9.1) and potential (9.2).*

Proof. Since $0 \leq H, H' < P$, one has $\Delta \geq -2^{23}$. Therefore

$$\Delta \log 2 \leq \frac{\Delta U_2 + 3 \cdot 2^{23}}{S}. \quad (9.11)$$

For $\Delta \geq 0$ use the upper bound on $\log 2$. For $\Delta < 0$, use the lower bound, multiply by the negative Δ , and use $-3\Delta \leq 3 \cdot 2^{23}$. Thus the uniform allowance is valid with either sign of the potential increment.

By Lemmas 7.1 and 9.1,

$$(A_{sgc}(q_s))_i \leq \frac{(T_s a)_i - 20^m E}{20^m Q} \leq -\frac{2^{37}}{Q}(\Delta U_2 + 3 \cdot 2^{23}).$$

The exact scale identity

$$\frac{2^{37}}{Q} = \frac{1}{80PS}$$

and (9.11) make this at most $-\Delta \log 2 / (80P)$. Finally

$$\frac{\Delta \log 2}{80P} = h(s^+(s, c)) - h(s) - \frac{7}{80} \log 2.$$

Rearrangement proves the desired Bellman inequality. $\square$

10 The certificate and completion of the proof

Encode the window by $s = \sum_{j=1}^{26} 2^{j-1} s_j \in \{0, \dots, 2^{26} - 1\}$, and encode an action by $r = 2s + c$. Its successor window is $s^+ = (2s + c) \bmod 2^{26}$. Thus the action range is exactly $0 \leq r < 2^{27}$. No symmetry reduction or restriction to typical inputs is used in the coverage argument.

There are 2^{26} states, two choices of the prepended bit, and 64 coordinates. Thus the certificate consists of $2^{26} \cdot 2 \cdot 64 = 2^{33}$ scalar inequalities. The following proposition is the computer-assisted step of the proof.

Proposition 10.1 (Concrete finite witness). *The code table of Appendix A satisfies (4.2). The potential H supplied in the electronic supplement has values in $\{0, \dots, 65535\}$, and all inequalities (9.10) hold for that potential at $m = 26$.*

Computer-assisted proof. The 64 Kraft inequalities follow from exact rational bounds on the exponential function, as detailed in Appendix B. The values of H are integers between zero and 65535 by their representation as 16-bit digits.

For this fixed H , the Lean certificate proves every instance of (9.10). The checks cover every state, both input bits, and every coordinate. They use integer arithmetic and the logarithm bounds established in Appendix C. To make the computation manageable, the implementation combines many scalar operations into operations on larger integers. Appendix E explains why this representation preserves each inequality and uses the same potential at every occurrence of a state.

The resulting finite proof establishes precisely the inequalities stated here. Appendix F identifies the formal theorems and their verification record; Appendix D gives the scalar calculation in ordinary mathematical notation. $\square$

Proof of Theorem 1.1. Proposition 10.1 and Lemma 9.2 establish the finite Bellman certificate. Equation (9.2) gives $0 \leq h \leq \log 2$. Apply Theorem 8.2 with (9.1). The coefficient simplifies exactly:

$$\frac{1-d}{1-13/20} \frac{(7/80) \log 2}{\log 2} = \frac{1-d}{4}.$$

The two finite-block correction terms become those in (1.1) and (1.2). Corollary 8.3 gives both the operational capacity bound and the arbitrary-length-sequence conclusion. $\square$

11 Consequences, limitations, and further work

The certificate produces a bound for every deterministic input word before any input distribution is chosen. It consequently allows arbitrary input dependence and nonstationary encoders. The only randomness needed for the channel model is independent deletion. No infinite-output test measure, stationary maximizing process, interchange of an extremum with a limit, or assumption of rate convergence enters the finite-block proof.

At the endpoint $d_0 = 13/20$, the rate coefficient is $7/80$ bits per input bit. The factor $1/(1-d_0) = 20/7$ gives the normalized coefficient $1/4$. The boundary correction is one bit for the potential and at most $\log_2(n+1)$ bits for the output-length partition bound. The decoding statement adds one further bit. These corrections are uniform in the window length once the potential range is fixed.

The size of the witness is a practical limitation. There are 2^{26} potential values and 2^{33} scalar inequalities, even though the predictive law itself has only 64 coordinates. The proof is finite and fully specified, but its independent reproduction is computationally substantial. A smaller potential representation or a sharper symbolic bound could reduce the cost without changing the channel argument.

A natural next question is whether a compact analytical potential or a richer predictive law can improve the coefficient. This paper establishes exactly $1/4$; it makes no claim that this coefficient is optimal.

Data and code availability

The electronic certificate, Lean proof, and reproduction instructions are available at <https://github.com/factorem/inv/bdc>. Appendix E.4 identifies the exact potential used in the proof.

A Exact code lengths

Table 1 gives the complete integer code table. Its columns are numerators with the common denominator $D = 36028797018963968 = 2^{55}$, not rounded decimal approximations to the lengths. Row i uses the context encoding (4.1); column c is the symbol being prepended. The table is extracted from `PackedCode.lean` and independently compared as rational numbers with the entries `w_i_c` in `FixedCode.lean` by the manuscript support script. Its extraction record is supplied in `evidence/paper-data-checks.json`.

Table 1: Exact code numerators; $w_{i,c} = N_{i,c}/2^{55}$.

i	$N_{i,0}$	$N_{i,1}$
0	8140985237798599	100381493485016752
1	66873931902426304	16802802218158854
2	19879035082725536	59583145120762928
3	82703681629426432	11839583596344262
4	16253725455866406	68340830244727080
5	39691610129809872	32607190098165324
6	13637566269366960	76217273187066688
7	85546870989978176	11136585844599798
8	14432245113150018	73652856759482000
9	30314556834137416	42449717367346168

Continued on next page

Table 1 continued

i	$N_{i,0}$	$N_{i,1}$
10	30330665508747488	42429379754296720
11	64538358005372064	17722545547055206
12	15011650986123108	71882938506889416
13	53177359053108936	23156126773920168
14	12102715404927432	81687641146737856
15	91429160876521520	9824405767541486
16	12255517331836568	81108917930130112
17	26180352673880852	48189955664327472
18	22362029444810328	54622984962053256
19	46365696235847392	27410516726943052
20	21709748934760232	55858540885961120
21	35746989920010232	36312140296164672
22	40492742859816456	31918081389777512
23	73858156317735360	14366713149911624
24	14701452053380894	72820548832190976
25	51128121200576008	24342698114409384
26	40211621795473736	32157647421268008
27	56657164459430400	21300509752417452
28	16141801286849876	68646935688668416
29	60542399330254304	19438124008426808
30	21275323130904828	56706926374105728
31	90541051759959888	10011100686904910
32	10179901224578372	89753600394574384
33	56473662083282152	21393702844091528
34	19177520510604684	61121727051520440
35	68067766045868232	16354331693356560
36	20143472611901056	59020023253528424
37	29911154338323744	42963717108079136
38	23657653026803248	52295606860683904
39	70552891808698168	15464707175748434
40	13884656710800832	75402036593447184
41	29606341183199600	43358185484296368
42	34161217403479312	37965987263737208
43	50572492398644336	24677437257282220
44	25009246527530016	50030991897680784
45	52116956249653944	23760906523063720
46	46524085925533016	27300810438840792
47	80672814765149840	12372093880466334
48	9873133277626622	91195587395410656
49	80814422296251424	12334104181822422
50	23452645434471648	52653206695368352
51	69943199371999512	15677655158896560
52	18700356812023332	62207262962683520
53	40162817189987136	32199483061127640
54	41494684372764712	31083385827777204
55	70910997599424808	15341190978643718
56	11541016200305046	83887581053610736
57	74495464092136672	14165441842135428
58	35465015548196632	36598760616713992
59	65828165595572704	17207511417398896

Continued on next page

Table 1 continued

i	$N_{i,0}$	$N_{i,1}$
60	12589933945592770	79870137884132624
61	58802627644078896	20246693290618700
62	17819301423540656	64300980547725592
63	100379572983432992	8141310872402782

B Rational proof of the Kraft inequalities

For rational x and an integer $n \geq 1$, define

$$P_n(x) = \sum_{j=0}^{n-1} \frac{x^j}{j!}, \quad R_n(x) = \frac{|x|^n(n+1)}{n!n}, \quad E_n^\pm(x) = P_n(x) \pm R_n(x).$$

For $|x| \leq 1$, the exponential series yields

$$E_n^-(x) \leq e^x \leq E_n^+(x). \quad (\text{B.1})$$

Indeed, after the first omitted term, every successive term has absolute ratio at most $1/(n+1)$. The absolute tail is bounded by

$$\frac{|x|^n}{n!} \sum_{k=0}^{\infty} \frac{1}{(n+1)^k} = R_n(x).$$

Set

$$\lambda_- = \frac{693147180559945309417232121458}{10^{30}}, \quad \lambda_+ = \frac{693147180559945309417232121459}{10^{30}}.$$

The exact rational comparisons $E_{32}^+(\lambda_-) \leq 2$ and $2 \leq E_{32}^-(\lambda_+)$, followed by monotonicity of the exponential, prove $\lambda_- \leq \log 2 \leq \lambda_+$.

Every code length in Table 1 lies in $[0, 3]$. For $w = w_{i,c}$ let $x = -w\lambda_-/4$, so $|x| \leq 1$. Then

$$2^{-w} \leq e^{-w\lambda_-} = (e^{-w\lambda_-/4})^4 \leq (E_{32}^+(-w\lambda_-/4))^4. \quad (\text{B.2})$$

The exponential enclosure is positive since it bounds $e^x > 0$ from above. The formal data supply rational $u_{i,c}$ satisfying

$$E_{32}^+(-w_{i,c}\lambda_-/4) \leq u_{i,c}, \quad u_{i,0}^4 + u_{i,1}^4 \leq 1.$$

Combining with (B.2) proves the Kraft row. These are finite rational comparisons; the stored $u_{i,c}$ are conveniences for the proof, not additional real-valued assumptions. They can be read verbatim in `FixedCode.lean`. The paper support script independently checks all 128 exponential upper comparisons and all 64 fourth-power row comparisons using rational arithmetic. It also directly checks

$$(E_{32}^+(-w_{i,0}\lambda_-/4))^4 + (E_{32}^+(-w_{i,1}\lambda_-/4))^4 \leq 1$$

for every row, so the printed code table and the stated Taylor formula suffice to reproduce the Kraft calculation without the auxiliary $u_{i,c}$. This auxiliary check is reported as a data check, not as a replacement for the Lean proofs.

Formal correspondence: `ExponentialBounds.lean`; `FixedCode.lean`; `PackedCode.lean`.

C The logarithm enclosure routine

This appendix specifies the finite logarithm data used in (9.8). It distinguishes the analytic enclosure, the integer grid, and the optimized affine lookup.

C.1 Positive centers and directed rounding

For $x, c > 0$ and $l \leq \log c \leq u$, the inequality $\log t \leq t - 1$ gives

$$l + \frac{x - c}{x} \leq \log x \leq u + \frac{x - c}{c}. \quad (\text{C.1})$$

For the lower inequality apply the elementary bound to c/x and negate; for the upper one apply it to x/c . Thus the residual can have either sign without changing the formulas.

The base table uses the 129 rational centers $c_j = 1 + j/128$, $0 \leq j \leq 128$, and integer endpoints L_j, U_j such that $L_j/S \leq \log c_j \leq U_j/S$. Each endpoint is connected to a rational exponential comparison using (B.1), and those comparisons are proved in the formal base table. A logarithm enclosure is not assumed merely because an external program computed it.

For any $a \geq 1$, exponent $k \geq 0$, and center j , put $\rho = 128a - (128 + j)2^k$. Equation (C.1), applied at $x = a/2^k$, gives the integer bounds

$$\ell_0(a; k, j) = k(U_2 - 3) + L_j + \left\lfloor \frac{S\rho}{128a} \right\rfloor, \quad (\text{C.2})$$

$$u_0(a; k, j) = kU_2 + U_j + \left\lceil \frac{S\rho}{(128 + j)2^k} \right\rceil. \quad (\text{C.3})$$

They satisfy $\ell_0/S \leq \log a \leq u_0/S$. The floor and ceiling here are mathematical signed floor and ceiling, including for negative ρ . The denominators are strictly positive. In particular, a center-selection algorithm need not be accurate for soundness: every center and exponent obey these inequalities. Selection affects only the width of the enclosure.

C.2 Normalized grid

For an integer $0 < a < 2^{144}$ choose the unique integer $K \geq 0$ with $2^K \leq a < 2^{K+1}$ and let

$$M = \left\lfloor \frac{a \cdot 65536}{2^K} \right\rfloor.$$

Then $65536 \leq M < 131072$ and

$$2^K \frac{M}{65536} \leq a < 2^K \frac{M + 1}{65536}. \quad (\text{C.4})$$

The normalized grid has 65,537 endpoints, including $131072/65536 = 2$. For its point $x_j = (65536 + j)/65536$, let L_j^f, U_j^f be the lower and upper integer bounds obtained from (C.2)–(C.3) at numerator $65536 + j$, exponent 16, and center index $\min(\lfloor j/512 \rfloor, 128)$, subtracting respectively $16U_2$ and $16(U_2 - 3)$. The formal implementation certifies its stored fine-grid lookup data against this specification. By monotonicity of the logarithm,

$$\ell_n(M, K) = K(U_2 - 3) + L_{M-65536}^f, \quad (\text{C.5})$$

$$u_n(M, K) = KU_2 + U_{M+1-65536}^f \quad (\text{C.6})$$

are lower and upper integer enclosures for $S \log a$. The upper endpoint uses $M + 1$, not M ; dropping that distinction would invalidate the bound for truncated mantissas.

C.3 Thirty-two affine pieces

Table 2 gives two arrays of packed integers C_j, V_j , $0 \leq j < 32$. Put

$$j = \left\lfloor \frac{M - 65536}{2048} \right\rfloor, \quad t = M \bmod 2048, \quad Z_2 = 14058695031755056497480129050576.$$

Define the packed pair

$$P_{M,K} = C_j + tV_j + 2^{64} \left\lfloor V_j/2^{64} \right\rfloor + KZ_2 \quad (\text{C.7})$$

and the integer endpoints

$$\ell(a) = (P_{M,K} \bmod 2^{64}) - 64, \quad (\text{C.8})$$

$$u(a) = \left(\left\lfloor P_{M,K}/2^{64} \right\rfloor \bmod 2^{64} \right) - 64. \quad (\text{C.9})$$

These equations and Table 2 fully specify the scalar enclosures used by the optimized checker. The bias of 64 keeps the packed fields nonnegative. Its removal in the raw-gradient expression is exact because $13 + 7 - 13 - 7 = 0$.

For all 65,536 normalized mantissas, the formal finite checks establish that the affine lower value at $K = 0$ is no larger than $\ell_n(M, 0)$ and the affine upper value is no smaller than $u_n(M, 0)$. They also bound each initial packed field below 2^{40} . The exponent extension adds the two appropriate integer endpoints for $\log 2$ without a carry between the fields for $K \leq 255$. Equations (C.4)–(C.6) then prove (9.6). Our argument range gives $K \leq 143$, within the proved extension range. The comparison is finite over mantissas but universal over all positive integer arguments in the stated range.

Formal correspondence: `LogArithmetic.lean;FixedLogTable.lean;FastLogTable.lean;FastLog.lean`.

Formal correspondence: `FineLogSpecification.lean;FineLogTable.lean;NormalizedLog.lean`.

Formal correspondence: `LinearLogData.lean;LinearLogTable.lean;BatchLogSound.lean`.

Table 2: Exact packed affine logarithm coefficients for (C.7).

j	C_j	V_j
0	3029110255766351240742567951	304723702261735822664128779
1	626973476636423485346604672103	295627465185037053085388339
2	1232299879142570971276078181558	287058565181173418458707781
3	1820087069255880127219887041878	278972398022974689088292508
4	2391322622363278445674133931514	271329321657426463147717584
5	2946913039194004356790734393508	264093881442419070878775134
6	3487692358087906409303199369574	257234293637913510724029067
7	4014429617625758289725741863820	250722039577571827716679498
8	4527835532524385841913716677158	244531367606667123322669097
9	5028568172134788765222313786618	238639034827666523506960442
10	5517238032226849728358099019438	233024011952325999380710151
11	5994412608490592564188146453934	227667132813552966719788163
12	6460620208205356964418398600853	222551020578429991126571262
13	6916353572940526883191379805296	217659792600309608677117211
14	7362072970654112404502366943658	212978931291605809954303432
15	7798208953905263875472675863143	208495173443329597790517298
16	8225164955827671911347441315920	204196288864160102753036743

Continued on next page

Table 2 continued

j	C_j	V_j
17	8643319380034990482345557886505	200071117273932730563133477
18	9053027779587140388379336067363	196109310049222130162350366
19	9454624546638279434554592189628	192301367116830341131604653
20	9848424624276205727343328918902	188638489379834204014775029
21	10234724977023008254414351955500	185112523377353139190046782
22	10613805873676990002616618146604	181715961284549146869911790
23	10985932185668302841952316483070	178441811785418291134307211
24	11351354505817082107913754575952	175283544732558478801960628
25	11710310079451300466046686077986	172235128040657606849493280
26	12063023765334560199628273506111	169290953899517267573213600
27	12409708959368355956599175925648	166445728093588306331807508
28	12750568261755467663416902264297	163694543788947116384544885
29	13085794196810361025276224982142	161032844639807491472176328
30	13415569934282221040373776930393	158456332554800257269175077
31	13740069885295466045168825898173	155960949250229197674185394

D A scalar specification of the exhaustive check

The following procedure is a readable specification of (9.10). Its variables are mathematical integers of unbounded size; logarithm endpoints are the integer functions of Appendix C. It is not the packed implementation used to accelerate the Lean proof.

1. Read the exact potential $H[0], \dots, H[2^{26} - 1]$ and the code numerators $N_{i,c}$. Require $0 \leq H[s] < 65536$.
2. For each integer s from 0 through $2^{26} - 1$, define $s_j = \lfloor s/2^{j-1} \rfloor \bmod 2$ for $1 \leq j \leq 26$. Start $q_i^{\text{int}} = 1$ and apply G_{s_j} for $j = 26, 25, \dots, 1$.
3. For each $c \in \{0, 1\}$, form $t = G_c q^{\text{int}}$ and $v_j = q_j^{\text{int}} + q_{j+32}^{\text{int}}$. Use (9.8) to obtain a .
4. Set $z = a$. For $j = 1, 2, \dots, 26$, replace z simultaneously by $T_{s_j} z$. “Simultaneously” means that all new coordinates use the old vector; an in-place coordinate loop without a copy is not this map.
5. Set $s' = (2s + c) \bmod 2^{26}$, compute $b(H[s], H[s'])$ from (9.9), and require $z_i \leq 20^{26} b(H[s], H[s'])$ for all $i = 0, \dots, 63$. Reject if any comparison fails.

The grouped proof establishes acceptance of this entire predicate through representation lemmas. It does not sample states. One full scalar implementation could independently check the same witness, but the manuscript support script does not claim to have rerun this 2^{33} -inequality procedure. Its purpose is to reproduce the paper’s tables and evidence snapshot; the supplied Lean development is the proof-producing verifier.

E Exact arithmetic and storage

E.1 Packed arithmetic

The production proof evaluates batches of 64 actions, with 64 coordinates per action. Each coordinate occupies a lane of width 256 in a natural number. For lane values z_k , define $\text{pack}(z) = \sum_k z_k 2^{256k}$. Addition and multiplication by a scalar have their coordinatewise interpretation whenever the

resulting lane values are below 2^{256} ; this follows by uniqueness of radix expansion. Natural subtraction agrees with coordinatewise subtraction when each subtracted lane is no larger than its corresponding minuend. The implementation proves these preconditions at each operation.

For the final comparison the sharper bounds are

$$0 \leq (T_s a)_i < 20^m 2^{113} < 2^{255}, \quad 0 \leq 20^m b(H, H') < 2^{255} \quad (m \leq 28).$$

The first follows by induction from $T_c a \leq 20 \max_i a_i$ and the certified initial bound. For the second, write $b = b_0 + b_1 H - b_1 H'$ with

$$b_0 = E + 2^{37}(7PU_2 - 3 \cdot 2^{23}), \quad b_1 = 2^{37}80U_2.$$

The exact integer inequalities $b_1 P \leq b_0$ and $b_0 + b_1 P < 2^{113}$ prove nonnegativity and the required upper bound, without allowing a truncated subtraction.

For $0 \leq A, B < 2^{255}$, the integer $B + 2^{255} - A$ lies strictly between zero and 2^{256} ; its bit at position 255 is one exactly when $A \leq B$. Applying this fact lane by lane proves that the packed comparison tests every scalar inequality in (9.10). It is not an average test, a floating-point tolerance, or a machine-word overflow heuristic. The kernel works with natural numbers, while the lane lemmas justify their intended vector interpretation.

Formal correspondence: `RawBatchGradient.lean; RawBatchBounds.lean; RawBatchSound.lean.`

Formal correspondence: `PackedArrayComparison.lean; PackedMatrixArithmetic.lean.`

E.2 A single potential across all batches

Store the potential as chunks

$$D_b = \sum_{j=0}^{31} H(32b + j) 2^{16j}, \quad 0 \leq b < 2^{21}.$$

Each $D_b < 2^{512}$. For a batch $r = 64b + k$, $0 \leq k < 64$,

$$\begin{aligned} s &= 32b + \lfloor k/2 \rfloor, \\ H(s) &= \left\lfloor D_b / 2^{16\lfloor k/2 \rfloor} \right\rfloor \bmod 65536, \\ H(s^+) &= \left\lfloor (D_{2b \bmod 2^{21}} + 2^{512} D_{(2b+1) \bmod 2^{21}}) / 2^{16k} \right\rfloor \bmod 65536. \end{aligned}$$

The last formula follows by writing the successor index $64b + k$ in chunks of 32 and then reducing it modulo 2^{26} . For $k < 32$ it reads the first chunk, and for $k \geq 32$ it reads the second. These identities prove that current and successor values are evaluations of one global potential, even where a batch or group boundary is crossed.

E.3 Finite coverage and the concrete proposition

There are 2^{21} batches, divided into $2^{10} = 1024$ groups of $2^{11} = 2048$ batches. Every action has a unique decomposition

$$r = 64(2048g + b) + k, \quad 0 \leq g < 1024, \quad 0 \leq b < 2048, \quad 0 \leq k < 64.$$

Existence and uniqueness follow from successive Euclidean divisions. Every action then has 64 coordinate obligations, for a total of $2^{27} \cdot 64 = 2^{33} = 8,589,934,592$ scalar inequalities.

E.4 Identification of the electronic certificate

The source potential file, relative to `upper_formal`, is

`results/small-windows/quantized-m26-howard.npy.`

Its SHA-256 is

`5e6818890131d589388c063634c3c975ef9c28c59962a932eaa1e38ec0ac3282.`

The array contains 2^{26} unsigned 16-bit values, a 128 MiB payload before its file header. The generated Lean data encode these values in chunks and lookup trees. The supplementary evidence snapshot identifies the source files and available round-trip checks. A hash identifies the byte sequence; it does not prove the inequalities. Logically, the Lean theorem is about the literal data it imports. Reproducing those data from the array is an additional provenance check, distinct from kernel verification of the imported literal witness.

The numerical search that proposed this witness is outside the proof dependencies. Neither convergence of a policy iteration nor accuracy of a floating-point eigenvalue calculation is assumed. A proposed potential is acceptable exactly when the integer checker and its proved soundness implication establish the required inequalities.

F Formal correspondence and verification record

F.1 The five closed declarations

The final file is `upper_formal/BDCFormal/Howard26Upper.lean`. Its declarations are:

1. `BDCFormal.upper_capacity_bound`: the operational capacity bound for all $13/20 \leq d < 1$;
2. `BDCFormal.upper_finite_block_bound`: (1.2) for every positive length and every block code;
3. `BDCFormal.upper_information_bound`: (1.1) for every law on the entire length- n input alphabet;
4. `BDCFormal.upper_sequence_bound`: the limit-rate converse for arbitrary input lengths tending to infinity;
5. `BDCFormal.upper_eventual_rate_bound`: (1.4), without a rate-convergence hypothesis.

The final capacity declaration has the following form:

```
theorem upper_capacity_bound :  
  ∀ d (hd : 13/20 ≤ d) (hd1 : d < 1),  
    operationalCapacity d (by linarith) hd1.le ≤ (1-d)/4 := by  
intro d hd hd1  
exact RawBatch.capacity_of_groups  
Howard26.groups Howard26.exhaustive d hd hd1
```

Its hypotheses are the deletion-parameter inequalities. The finite certificate is supplied by a proved term, not left as a hypothesis of the final theorem. The full body of the actual declaration, rather than this typeset excerpt, is the formal artifact. Appendix G maps the main mathematical steps to source modules.

F.2 Verification record

Verification completed on September 11, 2026. All 1,024 certificate groups compiled, and the analytic proof and 256 certificate components passed fresh replay. The final theorems were then checked over those verified imports. This was a modular verification, with each component replayed separately. The completed report and logs are preserved in `upper_formal/evidence/completed-audit/`.

The audit checks the theorem statements, their dependencies, the fixed channel definitions, and the identity of the source and compiled files. It also tests that deliberately introduced unproved assumptions are rejected. Commands for reproducing these checks are given in Appendix H; the detailed counts and results are in `verification.json` in the directory above.

F.3 Foundations and limits of the claim

The development uses Lean 4 [10] and the Mathlib library [11]. It is pinned to Lean 4.34.0-rc2 and Mathlib commit

d6893048e0d784c43f3cf098b61299b3a4b4aed0.

The audit permits the usual foundational dependencies `propext`, `Classical.choice`, and `Quot.sound`; it is designed to reject additional axioms and admission-based proofs. Large finite predicates use kernel evaluation and proved representation identities. External numerical optimization is not accepted as a proof of a real inequality. The completed audit checked this policy over the final dependency closure, including deliberate negative controls.

Formal verification establishes propositions about the specified mathematical definitions, subject to the proof assistant’s foundations and implementation. Matching those definitions to the intended channel is a separate semantic responsibility; the explicit model (2.1), the code definition, and the finite-block theorem make that responsibility inspectable. The formal guarantee is relative to these explicit definitions and the stated foundations.

G Correspondence between mathematics and Lean

All paths in this appendix are relative to `upper_formal/BDCFormal`. A module name identifies where the relevant definitions and proofs are developed; the exact final theorem names are in Appendix F.

Finite probability and the actual channel. `FiniteProbability.lean`, `FiniteEntropy.lean`, and `DeletionChannel.lean` define finite laws, entropy, retention masks, and the deletion channel. `Operational.lean` defines messages, encoders, decoders, average error, achievable rates, and operational capacity. These are the definitions summarized in Section 2.

Entropy and divergence. `LogSum.lean`, `DivergenceMixture.lean`, and `EntropyDataProcessing.lean` supply the support-sensitive log-sum, mixture, and information-contraction arguments of Section 3. `DecoderConverse.lean` connects uniform-message decoding error to divergence bounds.

Backward cost and predictive recursion. `BackwardCode.lean` builds the finite positive test laws and their partition bounds. `DeletionRecursion.lean` and `RewardRecursion.lean` relate prepending an actual input bit to the predictive filter and score increment. `FilterTangent.lean` proves the exact relative-entropy slack and global majorant.

Window and word certificates. `WindowReachability.lean` proves reachability with zero padding. `FiniteWindowConverse.lean` defines `FiniteWindowCertificate`, its coordinate interpretation, and the reduction to a word certificate. `WordPotentialConverse.lean` telescopes that certificate.

Thinning and operational consequences. `ChannelThinning.lean` proves the actual channel composition. `ThinningConverse.lean`, `FiniteBlockUpper.lean`, and `InformationBlockUpper.lean` establish the finite output-law, decoding, and information conclusions. `OperationalConverse.lean`, `SequenceConverse.lean`, and `SequenceUniformConverse.lean` provide the limiting results.

Exact real-to-integer bridge. `PackedCode.lean` connects the packed numerators to the rational code table through the identity `PackedCode.rational_eq`. `IntegerFilter.lean`, `RawGradient.lean`, and `RawBellman.lean` connect integer references, logarithm enclosures, and the signed potential correction to the real Bellman inequality. `RawBatchSound.lean` assembles the packed arithmetic lemmas. `RawBatchCapacity.lean` and `FiniteBlockGroups.lean` connect accepted global groups to the information-theoretic conclusions.

Global data and optimized evaluators. `ChunkPotential.lean` and `ChunkGroups.lean` prove the potential indexing and group coverage. The optimized evaluators are connected to the original checker in `FastCheck64Sound.lean`, `LiteralCheck64Sound.lean`, and their metadata and lookup support modules. The exhaustive data and group proofs are under `Howard26Data/` and `Howard26Checks/`. Their final assembly is `Howard26Upper.lean`.

H Reproduction and document provenance

In the computational release, `upper_formal` contains the formal project and `quarter_bound_paper` contains the manuscript sources, exact tables, evidence, and building utilities. The release includes every local proof dependency and the concrete literal data; pinned external Lean and Mathlib dependencies are obtained during setup. The root `README.md` distinguishes the completed audit evidence from the working directory of a fresh reproduction.

After installing the pinned Lean toolchain, initialize dependencies from `upper_formal` with

```
lake exe cache get
```

From the formal project directory, the assembly build is

```
lake build BDCFormal.Howard26Upper
```

To reproduce the explicit group-check registration and then the strict audit in an independent checkout, use

```
python3 scripts/run_howard_certificate.py --phase checks \
  --workers 4 --min-free-gib 3
python3 scripts/verify_upper.py --modular-fresh --require-upper
```

The toolchain and Mathlib revision must match the pinned versions. These commands are resource-intensive. A fresh checkout has no compiled certificate cache, so compilation and independent replay must both run. The manuscript build does not run them or alter the formal source tree.

To rebuild this paper from its recorded snapshot, run

python3 build_paper.py

To deliberately capture a new snapshot and regenerate the exact tables, run `prepare_paper.py` before building. The snapshot routine checks the expected code-table and Kraft relationships, records file hashes, and requires a completed verification report. The build script runs LaTeX sufficiently many times to resolve cross-references and rejects unresolved references or overfull boxes.

Scope of the source comparison. The paper is a mathematical exposition of the implemented chain, not a machine-checked translation of the prose itself. Its elementary log-sum and series arguments are written out for a human reader; the Lean implementation may use equivalent finite-law identities or library lemmas. The finite-block theorem, operational definitions, coefficient, state encoding, and integer predicate are matched explicitly. The new manuscript makes no change to the proof sources covered by the completed independent replay.

References

- [1] M. Cheraghchi and J. Ribeiro, “An overview of capacity results for synchronization channels,” arXiv:1910.07199, 2019. <https://arxiv.org/abs/1910.07199>.
- [2] M. Dalai, “A new bound on the capacity of the binary deletion channel with high deletion probabilities,” in *Proceedings of the IEEE International Symposium on Information Theory*, pp. 499–502, 2011. <https://doi.org/10.1109/ISIT.2011.6034177>. Extended preprint: <https://arxiv.org/abs/1004.0400>.
- [3] I. Rubinstein and R. Con, “Improved upper and lower bounds on the capacity of the binary deletion channel,” arXiv:2305.07156, 2023. <https://arxiv.org/abs/2305.07156>.
- [4] R. E. Blahut, “Computation of channel capacity and rate-distortion functions,” *IEEE Transactions on Information Theory*, vol. 18, no. 4, pp. 460–473, 1972. <https://doi.org/10.1109/TIT.1972.1054855>.
- [5] S. Arimoto, “An algorithm for computing the capacity of arbitrary discrete memoryless channels,” *IEEE Transactions on Information Theory*, vol. 18, no. 1, pp. 14–20, 1972. <https://doi.org/10.1109/TIT.1972.1054753>.
- [6] M. Pinto and J. Ribeiro, “Improved capacity upper bounds for the deletion channel using a parallelized Blahut–Arimoto algorithm,” arXiv:2604.05867, 2026. <https://arxiv.org/abs/2604.05867>.
- [7] T. M. Cover and J. A. Thomas, *Elements of Information Theory*, 2nd ed. Wiley-Interscience, 2006. <https://doi.org/10.1002/047174882X>.
- [8] J. Lin, “Divergence measures based on the Shannon entropy,” *IEEE Transactions on Information Theory*, vol. 37, no. 1, pp. 145–151, 1991. <https://doi.org/10.1109/18.61115>.
- [9] M. Rahmati and T. M. Duman, “Upper bounds on the capacity of deletion channels using channel fragmentation,” *IEEE Transactions on Information Theory*, vol. 61, no. 1, pp. 146–156, 2015. <https://doi.org/10.1109/TIT.2014.2368553>.
- [10] L. de Moura and S. Ullrich, “The Lean 4 theorem prover and programming language,” in *Automated Deduction—CADE 28*, pp. 625–635. Springer, 2021. https://doi.org/10.1007/978-3-030-79876-5_37.
- [11] The mathlib Community, “The Lean mathematical library,” in *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs*, pp. 367–381, 2020. <https://doi.org/10.1145/3372885.3373824>.